Test ID	Feature	Environment	Steps	Expected Result	Actual Result	Status	Notes/Defect
TP-001	Create Account	Node.js v25.3.0 + macOS 26.3	1. Run app 2. Select option 1 3. Enter name "Nik" 4. Enter initial deposit 1200	Account created with unique ID, balance $1200, one DEPOSIT transaction recorded	Account created successfully with ID, balance $1200, transaction logged	PASS	Works as expected
TP-002	Create Account - empty name	Node.js v25.3.0 + macOS 26.3	1. Select option 1 2. Enter empty string for name 3. Enter deposit 500	Should reject empty name or show warning	Account created with empty holder name, no validation	FAIL	No input validation on holder name. Empty names are accepted.
TP-003	Create Account - negative deposit	Node.js v25.3.0 + macOS 26.3	1. Select option 1 2. Enter name "Test" 3. Enter -500 for deposit	Should reject negative amount or show error	Account created with balance -$500.00	FAIL	No validation on initial deposit. Negative amounts are accepted, creating accounts with negative balances.
TP-004	Create Account - non-numeric deposit	Node.js v25.3.0 + macOS 26.3	1. Select option 1 2. Enter name "Test" 3. Enter "abc" for deposit	Should reject non-numeric input	Account created with balance NaN	FAIL	parseFloat("abc") returns NaN. No validation on input type. Balance becomes NaN and is corrupted permanently.
TP-005	View Account Details	Node.js v25.3.0 + macOS 26.3	1. Create account (note ID) 2. Select option 2 3. Enter the account ID	Should display account ID, holder name, balance, and creation date	Displays all fields correctly in a formatted box	PASS	Works as expected
TP-006	View Account - non-existent ID	Node.js v25.3.0 + macOS 26.3	1. Select option 2 2. Enter "ACC-9999" (non-existent)	Should show "Account not found" error	Shows "Account not found." in red	PASS	Works as expected
TP-007	List All Accounts	Node.js v25.3.0 + macOS 26.3	1. Create 2+ accounts 2. Select option 3	Should display table with all accounts, total count and total balance	Displays table correctly with totals	PASS	Works as expected
TP-008	List All Accounts - empty	Node.js v25.3.0 + macOS 26.3	1. Start with no accounts 2. Select option 3	Should show "No accounts found" message	Shows "No accounts found." in yellow	PASS	Works as expected
TP-009	Deposit Funds	Node.js v25.3.0 + macOS 26.3	1. Create account (note ID) 2. Select option 4 3. Enter account ID 4. Enter amount 3500	Balance increases by $3500, DEPOSIT transaction recorded	Balance updated correctly, transaction logged	PASS	Works as expected
TP-010	Deposit - negative amount	Node.js v25.3.0 + macOS 26.3	1. Create account with balance $1000 2. Select option 4 3. Enter account ID 4. Enter -100	Should reject negative deposit	Balance decreases by $100 (becomes $900). Negative deposit accepted.	FAIL	No validation. Negative deposit effectively works as a withdrawal, bypassing any withdrawal logic.
TP-011	Deposit - non-numeric input	Node.js v25.3.0 + macOS 26.3	1. Create account 2. Select option 4 3. Enter account ID 4. Enter "hello"	Should reject non-numeric input	Balance becomes NaN. All future operations on this account also show NaN.	FAIL	parseFloat("hello") = NaN. Once balance is NaN, it stays NaN forever — even depositing a valid number after gives NaN (because NaN + 100 = NaN). Account is permanently broken.
TP-012	Deposit - math expression input	Node.js v25.3.0 + macOS 26.3	1. Create account with $1000 2. Select option 4 3. Enter account ID 4. Enter "5000-125+(5*3)"	Should reject the input — a math expression is not a valid money amount	Balance becomes $6,000. The app reads only "5000" from the expression and silently ignores the rest. No error.	FAIL	parseFloat() reads digits from the start and stops at the first non-numeric character. "5000-125" becomes just 5000. The app should only accept plain numbers (like $50, $1000) — any non-numeric characters should be rejected.

Test ID	Feature	Environment	Steps	Expected Result	Actual Result	Status	Notes/Defect
TP-013	Withdraw Funds	Node.js v25.3.0 + macOS 26.3	1. Create account with $5000 2. Select option 5 3. Enter account ID 4. Enter 1000	Balance decreases by $1,000, WITHDRAWAL transaction recorded	Balance updated correctly, transaction logged	PASS	Works as expected for normal case
TP-014	Withdraw - overdraft (more than balance)	Node.js v25.3.0 + macOS 26.3	1. Create account with $1000 2. Select option 5 3. Enter account ID 4. Enter 6000	Should reject withdrawal or show insufficient funds	Withdrawal succeeds. Balance goes to -$5,000.00	FAIL	No overdraft protection. Balance can go negative without any warning. Code at line 243 just does `account.balance -= amount` with no check.
TP-015	Withdraw - negative amount	Node.js v25.3.0 + macOS 26.3	1. Create account with $1000 2. Select option 5 3. Enter account ID 4. Enter -500	Should reject negative withdrawal	Balance increases by $500 (becomes $1,500). Negative withdrawal accepted.	FAIL	No validation. Negative withdrawal effectively works as a deposit, bypassing deposit logic.
TP-016	Transfer - normal	Node.js v25.3.0 + macOS 26.3	1. Create two accounts (A and B) 2. Select option 6 3. From: A, To: B 4. Amount: 300	Money deducted from A, added to B, both get transaction records	Transfer works correctly, both accounts updated, transactions logged	PASS	Works as expected for valid transfer
TP-017	Transfer - to non-existent account	Node.js v25.3.0 + macOS 26.3	1. Create account A 2. Select option 6 3. From: A, To: "ACC-0000" (does not exist) 4. Amount: 200	Should show error "Recipient account not found"	Creates a NEW phantom account with the typed ID, empty holder name, and the transfer amount as balance	FAIL	Critical bug at lines 290-307. If recipient ID doesn't exist, a new account is silently created with no holder name. This is a security issue.
TP-018	Transfer - no overdraft check	Node.js v25.3.0 + macOS 26.3	1. Create account A with $100 2. Create account B 3. Transfer $5000 from A to B	Should reject transfer or show insufficient funds	Transfer succeeds. Account A balance goes to -$4,900	FAIL	Same as withdrawal — no balance check before deducting at line 279.
TP-019	Transfer - to self	Node.js v25.3.0 + macOS 26.3	1. Create account A 2. Select option 6 3. From: A, To: A 4. Amount: 100	Should reject self-transfer or show error	Transfer succeeds. Creates both TRANSFER_OUT and TRANSFER_IN on the same account.	FAIL	No validation to prevent self-transfers. Creates confusing duplicate transactions.
TP-020	View Transaction History	Node.js v25.3.0 + macOS 26.3	1. Create account, make deposits/withdrawals 2. Select option 7 3. Enter account ID	Should display table with all transactions	Displays transactions correctly in table format	PASS	Works as expected
TP-021	Delete Account	Node.js v25.3.0 + macOS 26.3	1. Create account (note ID) 2. Select option 8 3. Enter account ID	Account removed, no longer appears in list	Account deleted successfully	PASS	Works but has no confirmation prompt
TP-022	Delete Account - non-existent	Node.js v25.3.0 + macOS 26.3	1. Select option 8 2. Enter "ACC-9999"	Should show "Account not found"	Shows "Account not found." in red	PASS	Works as expected
TP-023	Delete account with balance	Node.js v25.3.0 + macOS 26.3	1. Create account with $5000 balance 2. Select option 8 3. Enter account ID	Should warn that account has remaining funds, or ask for confirmation	Account deleted immediately with $5,000 still in it. No warning about remaining balance.	FAIL	Money just vanishes. No check for non-zero balance before deletion. Real banks would never allow this.
TP-024	Invalid menu option	Node.js v25.3.0 + macOS 26.3	1. At main menu, enter "99" or "abc"	Should show error and return to menu	Shows "Invalid option. Please select 1-9."	PASS	Works as expected

Test ID	Feature	Environment	Steps	Expected Result	Actual Result	Status	Notes/Defect
TP-025	Data persistence	Node.js v25.3.0 + macOS 26.3	1. Create account 2. Exit app (option 9) 3. Restart app 4. List accounts	Account should still exist after restart	Account persists in bank-data.json	PASS	Works as expected
TP-026	SIGINT (Ctrl+C) data loss	Node.js v25.3.0 + macOS 26.3	1. Create account 2. Make a deposit 3. Press Ctrl+C instead of using Exit (option 9)	Should save data before exiting (like option 9 does)	Data from last operation may be lost. SIGINT handler at line 441-443 calls process. exit(0) WITHOUT calling saveData() first.	FAIL	Compare: exitApp() at line 388-393 DOES call saveData(). But Ctrl+C skips it entirely. Unsaved changes are lost.
TP-027	Zero amount operations	Node.js v25.3.0 + macOS 26.3	1. Create account 2. Deposit $0 3. Withdraw $0 4. Transfer $0	Should reject zero amounts as meaningless	All operations succeed with $0. Transaction records are created for $0 amounts, filling the history with meaningless entries.	FAIL	No validation to reject amount === 0. Creates noise in transaction history.